

LRU CACHE

LEAST RECENTLY USED

Felipe Marchi
Henrique Kato
Luiz Silva
Thiago Maciel

LEETCODE #146



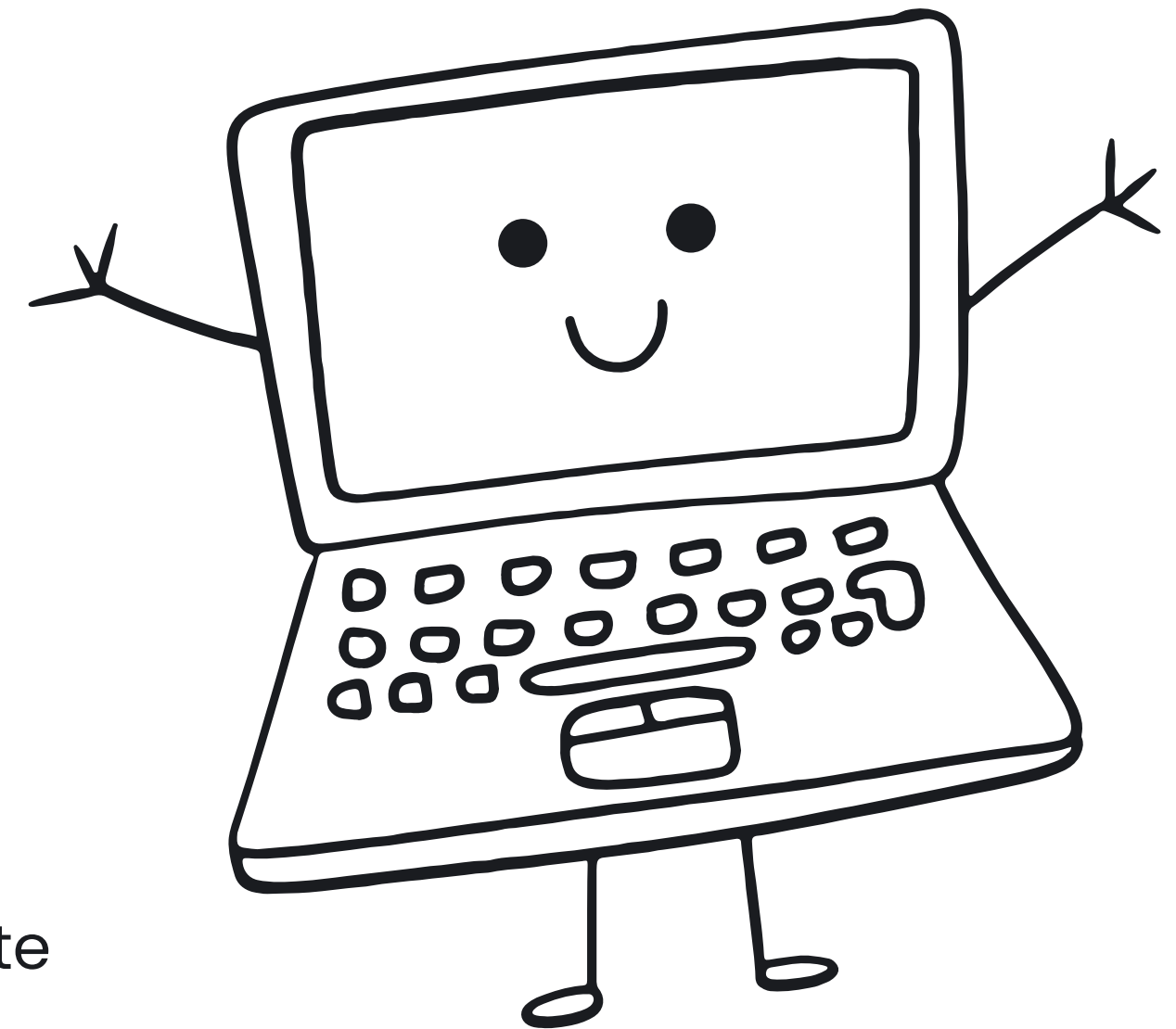
QUESTÃO

Projete uma estrutura de dados que implemente um cache LRU (**Least Recently Used**).

Implemente a classe LRUCache:

- **LRUCache(int capacity)**: inicializa o cache com capacidade máxima fixa.
- **int get(int key)**: retorna o valor da chave, se existir; senão, retorna -1. Essa operação conta como "uso".
- **void put(int key, int value)**: insere ou atualiza o valor. Se a capacidade for excedida, remove o item menos recentemente usado antes de inserir.

Restrição: get e put devem rodar em **$O(1)$** tempo médio.



EXEMPLO

Input

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]  
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, null, -1, 3, 4]
```

Passo a Passo

```
LRUCache cache = new LRUCache(2); // capacidade = 2  
cache.put(1, 1); // cache: {1=1}  
cache.put(2, 2); // cache: {1=1, 2=2}  
cache.get(1);    // retorna 1 → 1 vira o mais recente  
cache.put(3, 3); // capacidade excedida → remove chave 2 → cache: {1=1, 3=3}  
cache.get(2);    // retorna -1 (não existe mais)  
cache.put(4, 4); // remove chave 1 → cache: {3=3, 4=4}  
cache.get(1);    // retorna -1  
cache.get(3);    // retorna 3  
cache.get(4);    // retorna 4
```

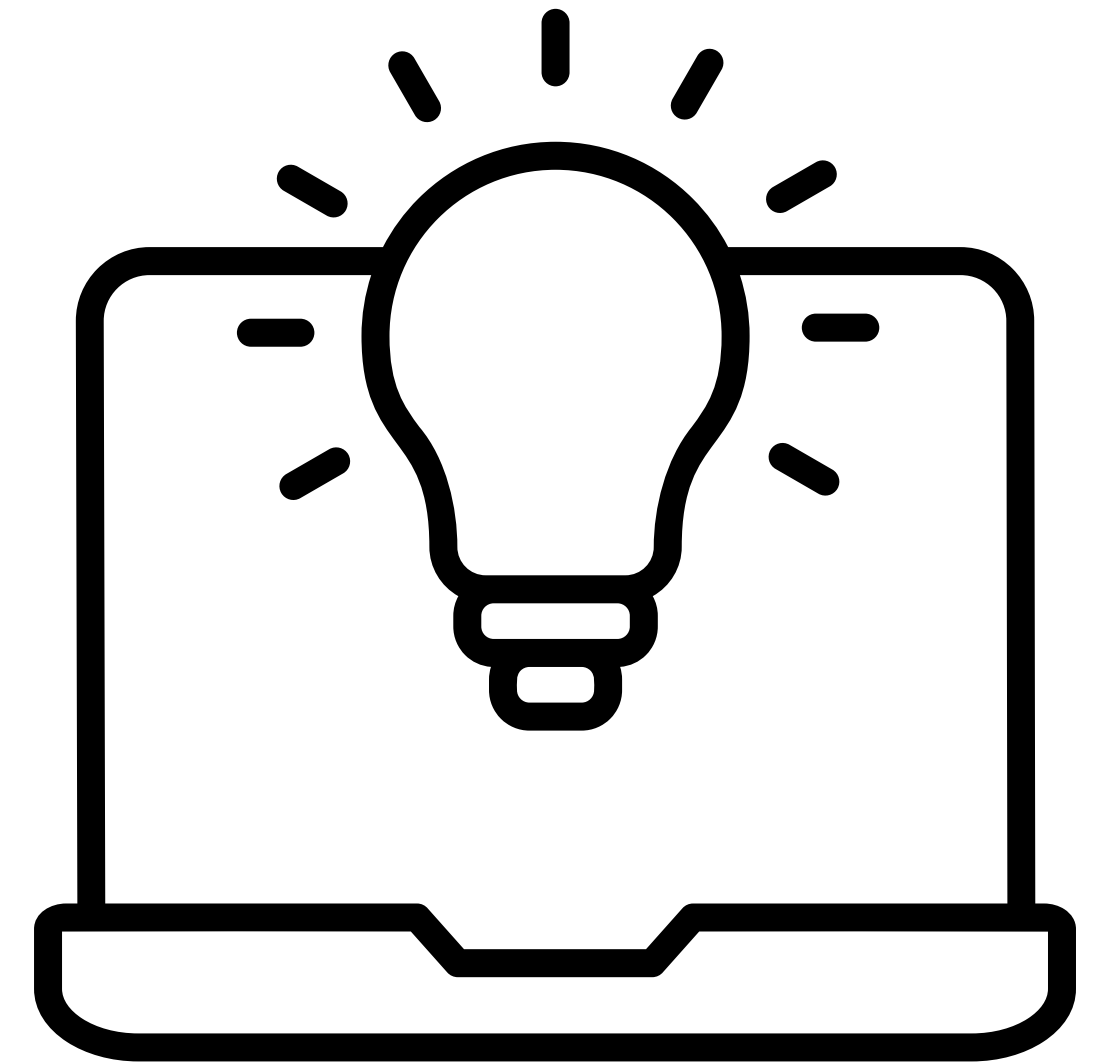
ESTRUTURA DE DADOS

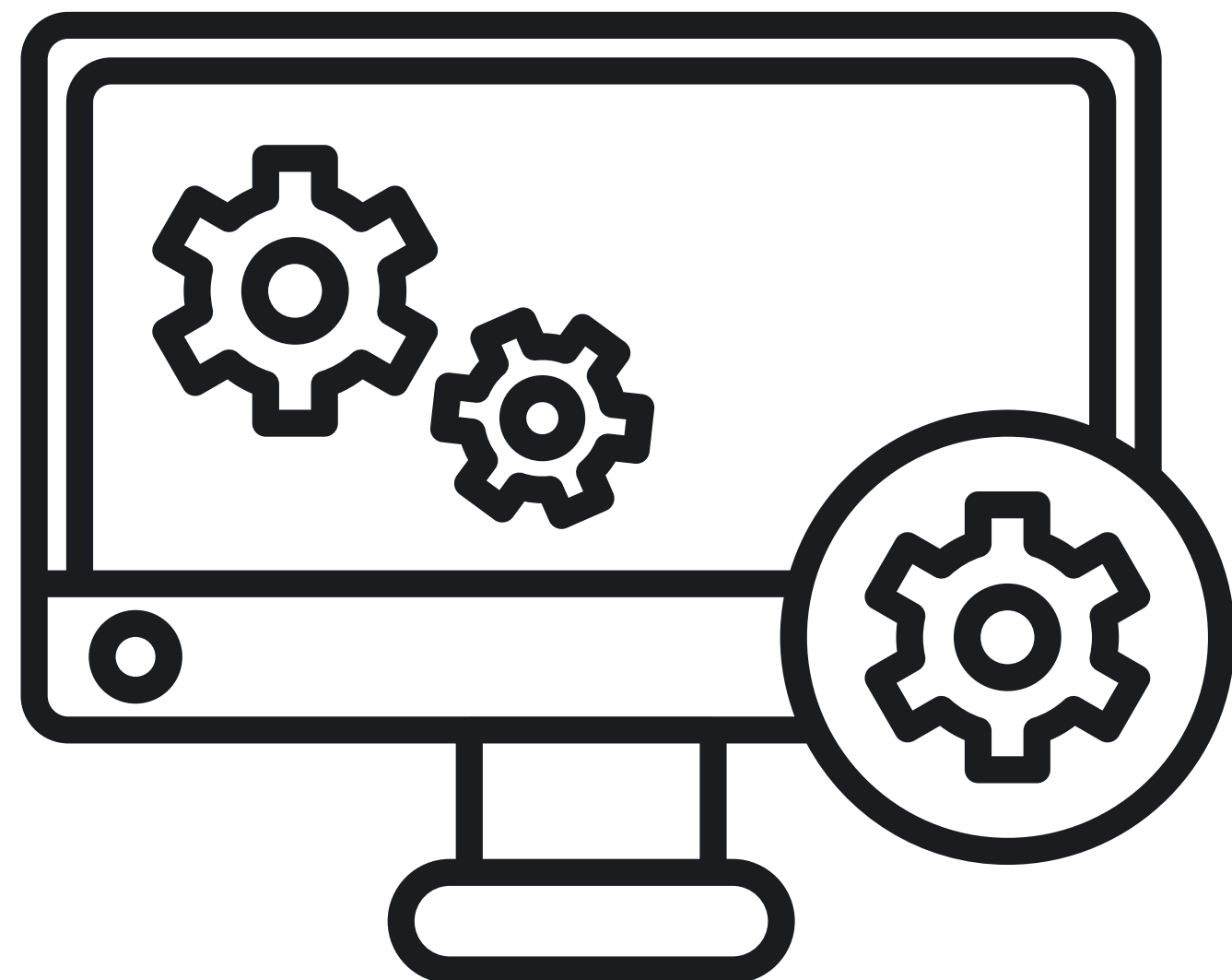
Diferente dos problemas clássicos de "ache o caminho mais curto" ou "ordene isso", aqui o enunciado pede diretamente:

"Projete uma estrutura de dados."

O desafio aqui é escolher quais estruturas combinar e como fazê-las trabalhar juntas para satisfazer uma restrição de tempo $O(1)$.

Esse exercício é usado em entrevistas de nível SDE1/SDE2 em Big Techs como Google, Amazon, Meta, Microsoft e Apple.

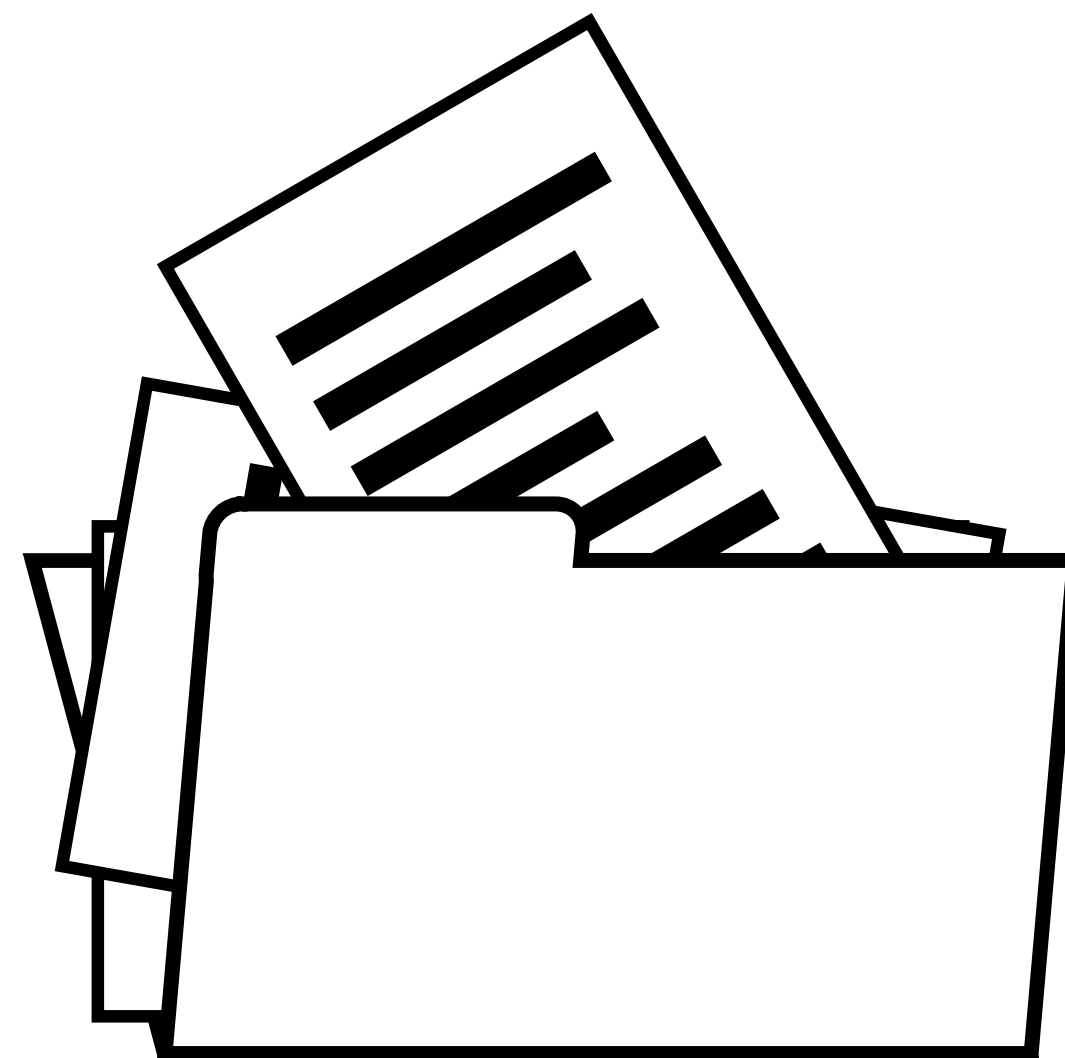




ABORDAGEM INCORRETAS

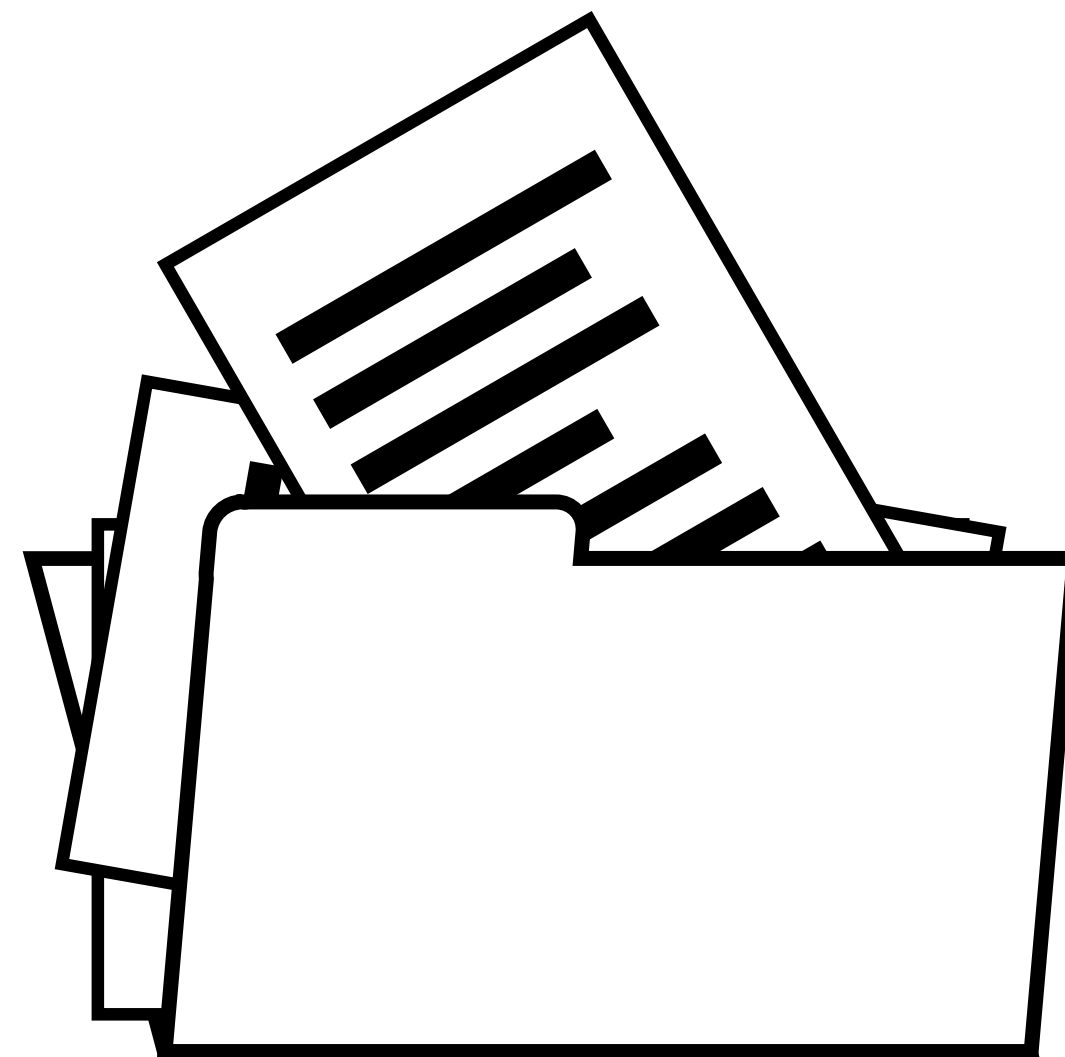
POR QUE SÓ A LISTA DUPLAMENTE ENCADEADA NÃO RESOLVE?

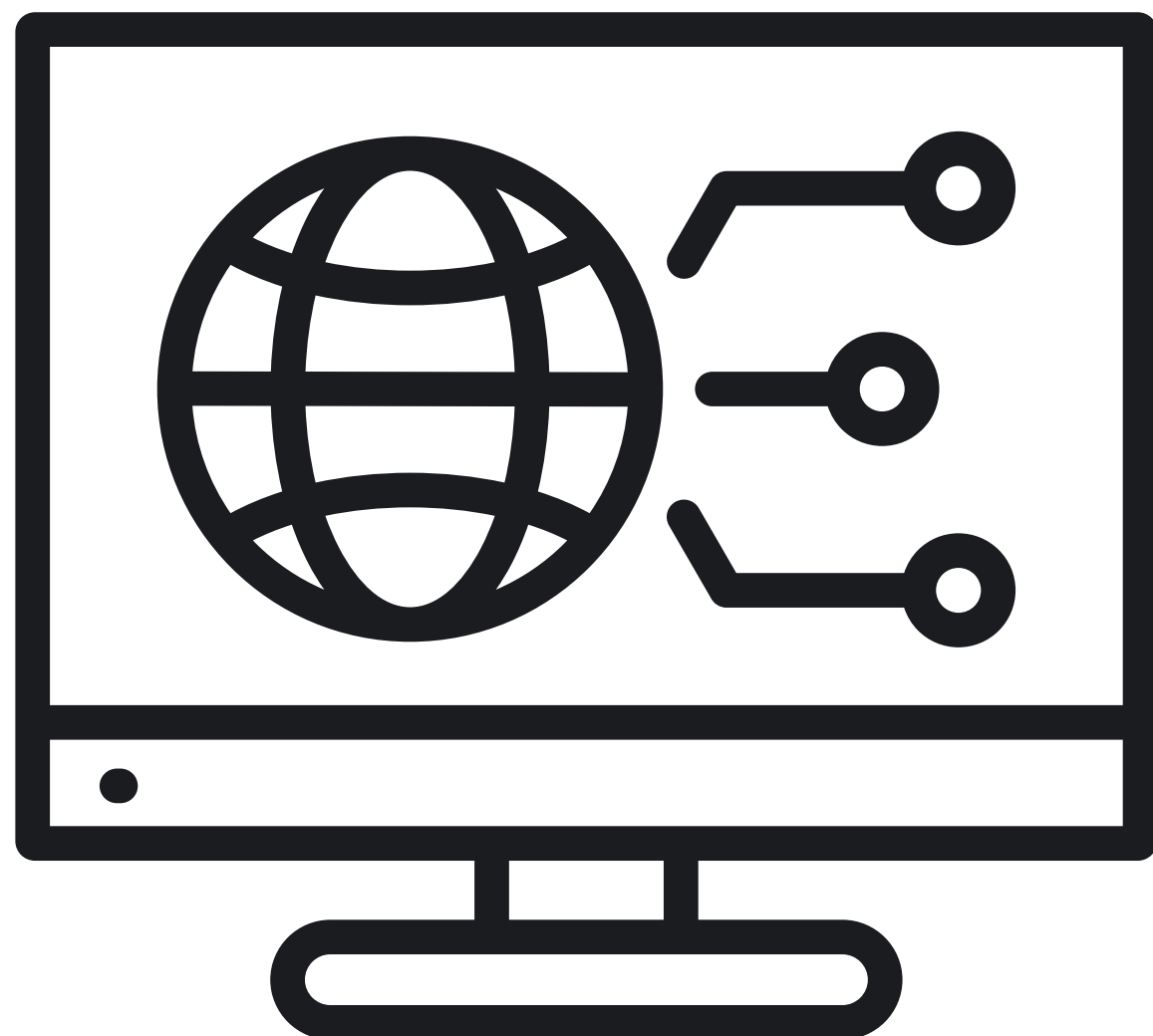
- Uma lista duplamente encadeada é ótima para manter a ordem de uso: mover um nó para o início ou remover o último nó custa $O(1)$ – considerando que já tenha o ponteiro do nó.
- **O problema:** para achar o nó de uma chave específica, ainda precisaria percorrer a lista inteira $\rightarrow O(n)$.
- **Ou seja:** ótima para ordenar, ruim para localizar.



POR QUE SÓ O HASH MAP NÃO RESOLVE?

- Um Hash Map é ótimo para localizar dados: buscar ou inserir o valor de uma chave específica: custa $O(1)$.
- **O problema:** ele não possui noção de ordem temporal: não sabe qual elemento foi acessado há 5 segundos e qual foi acessado há 1 hora.
- Para descobrir o item menos usado na hora da remoção, precisaria guardar timestamps e varrer todo o mapa buscando o menor tempo: voltando para o custo $O(n)$.
- **Ou seja:** Excelente para localizar, ruim para ordenar.





A IDEIA: COMBINAR DUAS ESTRUTURAS

Precisamos de duas coisas ao mesmo tempo:

- Achar qualquer item pela chave, rápido $\rightarrow$ Hash Map ($O(1)$)
- Saber qual item é o mais/menos usado e mover itens sem custo $\rightarrow$ Lista Duplamente Encadeada ($O(1)$)
- Solução: o Hash Map não guarda o valor — guarda o ponteiro para o nó na lista.
- chave $\rightarrow$ nó na lista (e o nó guarda o valor real)

COMO AS DUAS ESTRUTURAS TRABALHAM JUNTAS

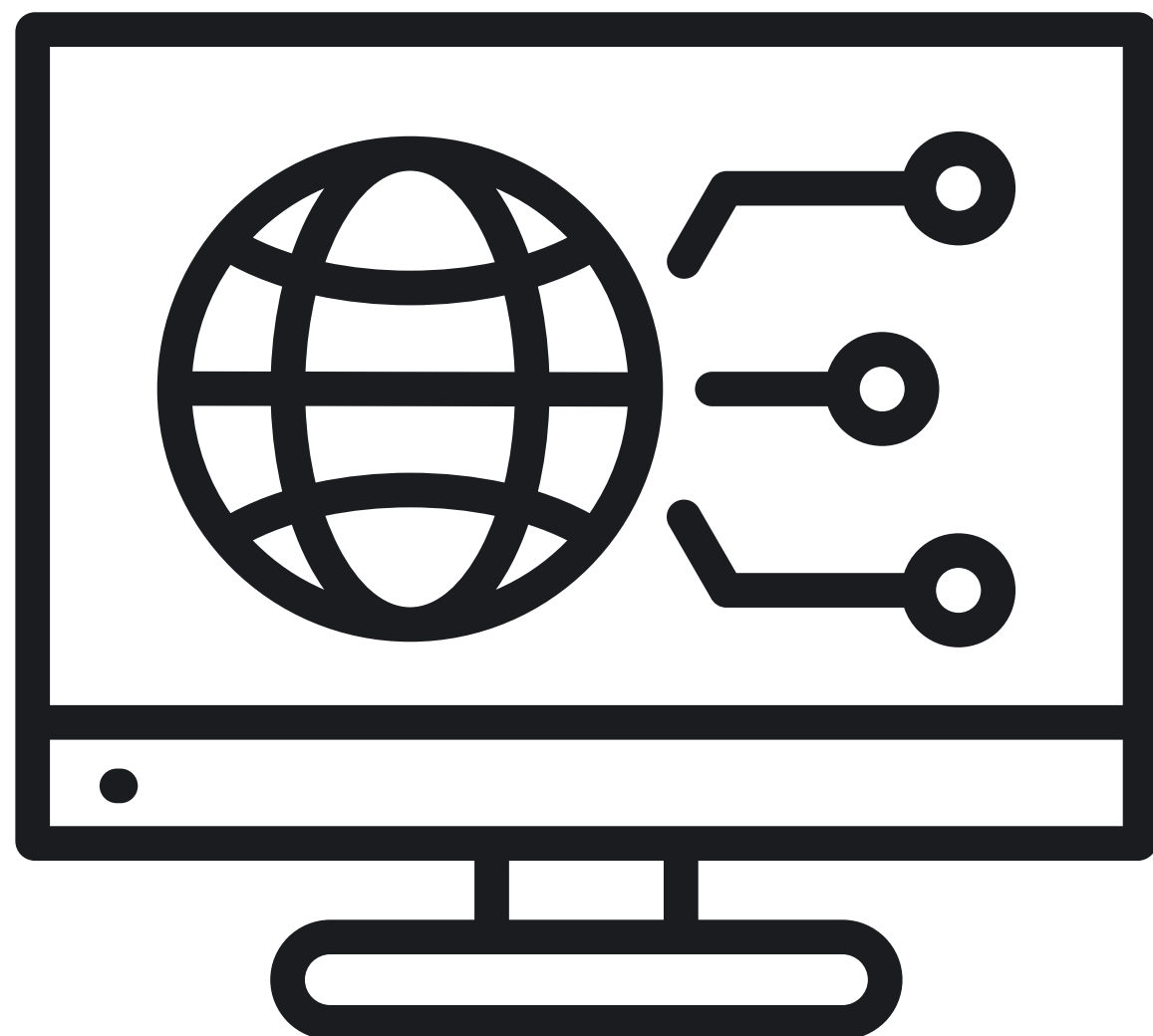
- Cabeça da lista = item mais recentemente usado
- Cauda da lista = item menos recentemente usado (próximo a ser removido)

Fluxo do **get(key)**:

- 1.Hash Map acha o nó em $O(1)$
- 2.Move esse nó para a cabeça (virou o mais recente)
- 2.Retorna o valor

Fluxo do **put(key, value)**:

- 1.Se cheio, remove o nó da cauda (e do Hash Map)
- 2.Cria novo nó, insere na cabeça, adiciona no Hash Map



CÓDIGO

```
def get(self, key):  
    if key not in self.map:  
        return -1  
    node = self.map[key]  
    self._remove(node)  
    self._add_to_front(node)  
    return node.value
```

ANÁLISE DE BIG-O E APLICAÇÕES



ANÁLISE DE COMPLEXIDADE

Abordagem	get	put	Espaço	Limitação
Lista / Array	$O(n)$	$O(n)$	$O(n)$	Precisa percorrer os elementos
Apenas Hash Map	$O(1)$	$O(1)$	$O(n)$	Não mantém a ordem de uso
Hash Map + Lista Duplamente Encadeada	$O(1)$	$O(1)$	$O(n)$	Usa memória extra

Principal trade-off: utilizamos mais memória para obter operações muito mais rápidas.

ONDE O CONCEITO DE LRU É UTILIZADO?

- Navegadores
- Sistemas Operacionais
- Bancos de Dados
- Caches de Aplicações
- Aplicações e APIs